MODULE NAME:	MODULE CODE:
CLOUD DEVELOPMENT A	CLDV6211/w

ASSESSMENT TYPE: POE (PAPER AND MARKING RUBRIC)

TOTAL MARK ALLOCATION: 300 MARKS

TOTAL HOURS: A MINIMUM OF 45 HOURS IS SUGGESTED TO COMPLETE THIS ASSESSMENT

By submitting this assignment, you acknowledge that you have read and understood all the rules as per the terms in the registration contract, in particular the assignment and assessment rules in The IIE Assessment Strategy and Policy (IIE009), the intellectual integrity and plagiarism rules in the Intellectual Integrity and Property Rights Policy (IIE023), as well as any rules and regulations published in the student portal.

INSTRUCTIONS:

1. ***No material may be copied from original sources, even if referenced correctly, unless it is a direct quote indicated with quotation marks. No more than 10% of the assignment may consist of direct quotes.***
2. ***Please ensure that you submit your assignment through Turnitin. Please make sure you attach a similarity report to your POE if you are required to submit a hard-copy of your PoE.***
3. ***Make a copy of your assignment before handing it in.***
4. *Assignments must be typed unless otherwise specified.*
5. *Begin each section on a new page.*
6. *Follow all instructions on the PoE cover sheet.*
7. *This is an individual assignment.*

© The Independent Institute of Education (Pty) Ltd 2026

Referencing Rubric

Providing evidence based on valid and referenced academic sources is a fundamental educational principle and the cornerstone of high-quality academic work. Part of achieving this quality is referencing in a way that is consistent and congruent with the requirements of the referencing style being used.

Therefore, inconsistent and/or incongruent referencing will result in a penalty of **a maximum of ten percent being deducted from the overall percentage** awarded to your assessment submission.

Please note that **evidence of plagiarism** in the form of copied or unreferenced work, absent reference lists, or exceptionally poor referencing **may result in action being taken in accordance with The IIE's Intellectual Integrity and Property Rights Policy (IIE023)**. Similarly, **evidence of excessive AI usage may result in action being taken in accordance with The IIE's Student Conduct, Discipline and Safety Policy (IIE015)**.

Markers are required to provide feedback to students by **circling/underlining the information in the table below that best describes the student's work and by adding constructive commentary where appropriate.** The examples provided are not exhaustive but illustrate the errors.

Deductions

- Where the student's work contains **five or more errors** aligned to the **minor errors column** below, **deduct 5% from the overall percentage.**
- Where the student's work contains **five or more errors** aligned to the **major errors column** below, **deduct 10% from the overall percentage.**
- **Where both minor and major errors** (e.g. two minor and three major, etc.) are present, **deduct 10% only** (and not 5% or 15%) from the overall percentage.

Required: **Consistent and congruent referencing**	**Minor errors** **Deduct 5% from overall percentage.** **Example: if the response receives 70%, deduct 5%. The final mark is 65%.**	**Major errors** **Deduct 10% from the overall percentage.** **Example: if the response receives 70%, deduct 10%. The final mark is 60%.**
Consistency • The **correct referencing style** for the **discipline** – i.e., either **Harvard**, OR **APA** (for Psychology), OR **Law**, OR **IEEE** (for ICT/Engineering) – has been used consistently for all in-text references and in the bibliography/reference list. • **Concepts** and **ideas** that are **quoted** and/or **paraphrased** are referenced consistently throughout. • **Position of the in-text reference**: an in-text reference is positioned consistently where appropriate for every quote and paraphrase.	**Minor inconsistencies:** • The referencing style used is generally consistent with what is required, but there are one or two changes/errors in the format of in-text referencing and/or in the bibliography/reference list. • For example, page numbers for direct quotes in-text have been provided for one source, but not in another. Or, two book chapters in the bibliography/reference list have been referenced in two different formats. Or, the publication year has been placed after the author name in one bibliography/reference list entry, and after the source title in another, etc. • Concepts and ideas in quotes and/or paraphrases are typically referenced, but a full in-text reference is missing or incomplete from one or two small sections of the work. • Position of the references: in-text references are only given at the beginning and/or end of every paragraph.	**Major inconsistencies:** • Poor and wholly inconsistent referencing style used in-text and/or in the bibliography/reference list. • Multiple referencing styles for the same source types have been used. • For example, the format for direct quotes in-text and/or book chapters in the bibliography/reference list and/or year of publication in the bibliography/reference list is different across multiple instances. • Concepts and ideas in quotes and/or paraphrases are haphazardly referenced in-text. • Position of the references: in-text references are only given at the beginning or end of large sections of work.
Feedback on referencing consistency:		
Congruency • **Each source** reflected within **in-text references** is included accurately in the **bibliography/reference list.** • All **bibliography/reference list** entries are in the **required order** for the referencing style used (e.g. alphabetical, alphabetical under subheadings, numerical). • All **direct quotes** and **paraphrases** have been **integrated appropriately** into the text using introductory phrases, accurate grammar, etc.	**Minor incongruences:** • There is largely a match between the sources presented in-text and those in the bibliography/reference list, but one or two sources that appear in-text do not appear in the bibliography/reference list, or vice versa. Or key source information is missing from one or two in-text references or bibliography/reference list entries only (e.g. publication year, city of publication, URL date accessed, etc.). • There is a clear and largely accurate ordering of sources in the bibliography/reference list as required by the referencing style used, but with one or two references out of order. • An attempt has been made for source integration into the text using appropriate introductory phrases and grammar, but one or two quotes or paraphrases do not flow as clearly or logically within the sentence structure as they could.	**Major incongruences:** • No relationship/several incongruencies between the in-text referencing and the bibliography/reference list. • For example, multiple sources are included in-text, but not in the bibliography, and/or vice versa. Key source information is missing from multiple in-text references and/or reference list entries. A URL link, rather than the actual reference, is provided in the bibliography. Sources are repeated in the reference list, etc. • Most sources are listed in a haphazard order throughout the bibliography/reference list. • Few to no appropriate introductory phrases or rules of grammar have been applied, and many direct quotes and/or paraphrases feel disconnected from the flow of the text.
Feedback on referencing congruency:		
Overall feedback on referencing, with suggested improvements:		

© The Independent Institute of Education (Pty) Ltd 2026

Background

EventEase, an up-and-coming event management company, coordinates a variety of events across different venues. Due to rapid growth and increasing demand, EventEase has been overwhelmed with managing multiple venue bookings, scheduling, and handling booking conflicts. Currently, all scheduling and bookings are managed manually, leading to double bookings, last-minute cancellations, and a lack of visibility into venue availability.

EventEase has contracted your development team to build an efficient, user-friendly online booking platform to streamline these processes. This application should simplify venue management, prevent booking conflicts, provide a clear view of scheduled events, and enable guests to book venues based on availability. The app should also offer search and filtering capabilities to make it easy to view and manage bookings.

The first phase of the project is building the admin platform, so customers will not be able to make bookings themselves; they can email, call, or walk in and request to make a booking, and from there, one of the booking specialists will use the system to make the booking for the customer. An event can be loaded onto the system prior to a venue becoming available, so that once the venue does become available, the booking can be made.
The CEO of EventEase has provided the following requirements that must be present for the new Venue Booking System:

- EventEase needs a user-friendly web application that allows the booking specialists to browse available venues, view events, and make or manage bookings.
- The system should allow authorised personnel to create, view, update, and delete venue and event information, with secure handling of image uploads for each venue.
- Prevent double bookings for venues and restrict deletion of venues or events associated with existing bookings.
- Ensure that all booking information is stored securely (initially locally, eventually in the cloud).
- Some important information to be stored includes the locations, capacity, venue/event names, start & end dates, and unique booking IDs.

EventEase envisions a future where this platform integrates with cloud services, allowing seamless scaling as its needs grow.

© The Independent Institute of Education (Pty) Ltd 2026

Introduction

You will build a Venue Booking System for EventEase using ASP.NET Core MVC. You will adopt a professional "Dev-to-Prod" workflow, starting with a local development environment and culminating in a live cloud deployment.

The application will follow three main development phases, each building upon the previous phase.

Development Phases:

Part 1: Project Foundation and Local Development

Database Design:

- Develop an Entity-Relationship Diagram (ERD) that includes the Venue, Event, and Booking tables.
- You may add additional tables, but keep the structure simple.

Initial Application Development:

- Create an ASP.NET Core MVC project with models for Venues, Events, and Bookings.
- Build the application with the necessary controllers, views, and initial CRUD functionality.
- **Database**: Configure the application to use SQL LocalDB (or SQL Express) for local data persistence.
- For images, use placeholder URLs (links to sample images) for venues and events in this phase

Local Execution:

- Ensure the web app runs successfully on localhost (IIS Express or Kestrel).
- Ensure data persists in the local database between sessions.

Part 2: Enhancing Functionality and Local Cloud Emulation

Error Handling and Validation:

- Implement validation rules to prevent double bookings.
- Add logic to restrict the deletion of venues/events associated with active bookings.

© The Independent Institute of Education (Pty) Ltd 2026

Image Management with Local Blob Emulation:

- Replace placeholder URLs with image upload functionality.
- Instead of connecting to live Azure Storage, configure the application to use the Azurite Emulator (Local Blob Storage).
- Store images in a local container named venue-images.

Enhanced Display and Search:

- Develop a consolidated view to display bookings effectively.
- Add a search feature to browse venues, events, and bookings.

Part 3: Advanced Filtering and Cloud Migration (Live Deployment)

Filtering and Event Type Integration:

- Add a new EventType lookup table.
- Enhance the search functionality to filter results by event type, venue, date range, and availability.

Go Live - Migration to Azure:

- This is the "Production" phase. You will migrate your local environment to the live cloud.
- Publish the Web App to Azure App Service.
- Migrate your LocalDB data to a real Azure SQL Database.
- Switch from Azurite to a real Azure Storage Account for images.

Reflective Report:

- Submit a report reflecting on the development journey and the specific process of migrating from a local environment to a live cloud environment.

© The Independent Institute of Education (Pty) Ltd 2026

Instructions

This Portfolio of Evidence (POE) requires you to create the Web Application, Database Storage, and Storage components for EventEase, progressively developed through Part 1, Part 2, and culminating in the final POE solution.

To work on the POE, students are required to:

- Install Microsoft Visual Studio 2022 (Ensure the "Azure Development" workload is ticked during installation to include the Azurite Emulator).
- Install Azure Storage Explorer (Free desktop tool) to manage local blob storage.
- Have access to an Azure account (Note: This is strictly required for Part 3 only).

The submission of each part of the POE will require you to do the following:

- Create a document in MS-Word which contains the following:
  - Your ST number.
  - The module code.
  - The URL to your GitHub repository source code.
    - As for all your programming modules, if your work is not on GitHub, and a link is not added to arc, you will lose 5% off your hand in.
  - YouTube video showing full details of your application running and explaining your work. This is needed for any marks to be given
  - **Part 3 Only:** The URL of the deployed Web App.
  - **Part 3 Only:** Showing in screenshots that your resources have been deleted

All answers required, including typed answers, diagrams, and screenshots.

The document name must follow the format shown here: **StudentNumber_ModuleCode_Part#.**

Submit this document in Arc, using the submission link on the Arc page for this module.

© The Independent Institute of Education (Pty) Ltd 2026

Part 1 — Project Foundation and Initial Deployment (Marks: 100)

Related Content: Learning Units: 1-2

At the end of this specific part, students should be able to:

Practically

- *Design and create a Basic Database Structure.*
- *Implement ASP.NET Core MVC Basics.*
- *Configure a Local SQL Database connection (SQL LocalDB).*
- *Run a data-driven web application locally.*

Theoretically

- *Explain the difference between deploying on-premises and in the cloud.*
- *Identify key differences between Azure hosting models.*

A. Design the EventEase Database

Based on the background information provided earlier, develop an Entity-Relationship-Diagram with an accompanying Database Script that will hold the data to be captured into the EventEase system. Your database should contain three (3) main tables for now: Venue, Event, and Booking.

B. Develop the Web Application

Build an ASP.NET Core web application with corresponding models, controllers, and views for the EventEase system.

Your web application should include the following:

- The relevant Models, Controllers, and Views for each of the Venues, Events & Bookings.
- Your application should communicate with **SQL LocalDB** (or SQL Express), implementing CRUD functionality.
- For the images, use placeholder URLs (links to sample images) for venues and events in this phase.

C.Local Persistence Setup

- Ensure your appsettings.json is configured to point to your local database instance.
- Verify that you can stop and start the application without losing data.

© The Independent Institute of Education (Pty) Ltd 2026

D. Cloud Computing Basics

1. In what ways does deploying an application in the cloud differ from deploying it on-premises, particularly regarding security, deployment speed, and resource management? Use examples to illustrate your points.
2. What are the key differences between Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS)? Why might EventEase benefit from the use of PaaS over the other two when eventually moving to the cloud?

Submission Notes

- Ensure all the relevant material is present in your submission, i.e., your locally developed web app, ERD, Database Script, Word document, etc.
- Provide screenshots of the application running on localhost.
- Include your GitHub repo; if not submitted to GitHub, it will result in a 5% off your assignment.
- Provide a detailed YouTube video going over the complete assignment, remember your lecturer might not be able to connect to the same database as yours, so please make sure it is very detailed and shows what is required in the rubric and brief in your video.

© The Independent Institute of Education (Pty) Ltd 2026

Part 2 — Enhancing Functionality and Integrating Cloud Storage (Marks: 100)

Related Content: Learning Units: 3-6

At the end of this specific part, students should be able to:

Practically

- *Implement Error Handling and Validation.*
- *Manage Images with **Azurite** (Local Blob Storage Emulator).*
- *Enhance Application Interface and Display.*
- *Implement a Basic Search Functionality.*
- *Manage Configuration for Development Environments.*

Theoretically

- *Explain how Azure Cognitive Search differs from traditional search engines.*
- *Discuss the importance of normalisation in database design.*
- *Compare the differences between relational and NoSQL databases.*

- **Integrate Local Blob Storage (Azurite)**

In Part 1, your event and venue images were being saved as simple URLs. For Part 2, your images must be stored using the **Azurite Emulator** to simulate Azure Blob Storage without using cloud credits.

- Update your appsettings.json to use the development storage connection string: UseDevelopmentStorage=true.
- Implement code to upload images to a container named venue-images.
- Use **Azure Storage Explorer** to verify the images are being saved locally.

B. Implementing Error Handling and Validation

We want to improve the User Experience by implementing validation rules and proper error handling.

- Add validation to prevent the double booking of a venue for the same date/time.
- Add validation to restrict the deletion of venues or events associated with active bookings.
- Ensure the application does not crash on common user error actions.
- Display an alert to the user when a validation error occurs.

© The Independent Institute of Education (Pty) Ltd 2026

C. Enhanced Display and Search:

In Part 1, your database included only 3 tables. Now, to enhance the booking display, you are required to incorporate a view that consolidates the relevant booking information.

- This view should include all the relevant information from the venue and event tables.
- Add a simple search function to this new booking's view so the booking specialists can search for bookings via the bookingID or Event Name.

D. Deployment Verification (Local)

- Submit screenshots of **Azure Storage Explorer** showing your local venue-images container and the uploaded files.
- Submit screenshots of the application displaying these uploaded images.

E. Database Design, Cognitive Search

- Explain how Azure's Cognitive Search differs from traditional search engines and discuss potential use cases where Cognitive Search would offer a clear advantage.
- Why is database normalisation important? Discuss the impact of both normalised and denormalised structures on performance and scalability.

Submission Notes

- Ensure all the relevant material is present in your submission.
- Include screenshots of Azure Storage Explorer verifying local blob storage.
- Remember that this submission must include code attribution & traditional referencing as it includes both practical and theoretical content.
- Provide screenshots of the application running on localhost.
- Include your GitHub repo; if not submitted to GitHub, it will result in a 5% off your assignment.
- Provide a detailed YouTube video going over the complete assignment, remember your lecturer might not be able to connect to the same database as yours, so please make sure it is very detailed and shows what is required in the rubric and brief in your video.

© The Independent Institute of Education (Pty) Ltd 2026

Part 3 — Advanced Filtering, Reporting, and Documentation (Marks: 100)

Related Content: Learning Units: 7-9

At the end of this specific part, students should be able to:

Practically

- *Enhance Search with Filtering and Event Type Classification.*
- *Migrate local resources to live Azure Cloud resources.*
- *Deploy an ASP.NET Core Web App to Azure App Service.*
- *Finalise Documentation and Reflection.*

Theoretically

- *Discuss how Cosmos DB differs from traditional databases.*
- *Discuss key considerations when designing logic apps that handle sensitive data.*
- *Explain how combining Event Grid with other services can create robust workflows.*

A. Advanced Filtering

Extend the search feature implemented in Part 2:

- Add a new EventType lookup to your database with predefined categories.
- Implement filters in your search functionality to filter results by event type, date range, and venue availability.

B. The "Go Live" Migration

Now you must move your application from your local environment to the live Azure Cloud.

- Database: Create an Azure SQL Database. Migrate your schema and data from your LocalDB to Azure SQL.
- Storage: Create an Azure Storage Account. Update your application configuration to point to the live storage keys instead of the local emulator.
- Web App: Create an Azure App Service and publish your application.
- Configuration: Ensure your deployed app is reading from the live database and live storage, not your local machine.
- Drop all the resources and show proof

© The Independent Institute of Education (Pty) Ltd 2026

C. Reflective Technical Report

Write a reflective report documenting your development journey. Specifically, focus on the transition from Part 2 (Local) to Part 3 (Cloud).

Your report must include:

- A detailed description of the application's full feature list.
- **Component Discussion**: Which Azure services were used and why?
- The Migration Experience: Reflect on the process of changing from LocalDB/Azurite to Azure SQL/Storage. What configuration changes were required? Why is this separation of environments important in professional development?
- Discuss the technologies used to build the project and why they were used.

Submission Notes

- Ensure all the relevant material is present in your submission.
- Do not forget to provide any relevant URLs, such as the web app URL.
- Remember that this submission must include code attribution & traditional referencing as it includes both practical and theoretical content.
- Remember that this submission must include code attribution & traditional referencing as it includes both practical and theoretical content.
- Provide screenshots of the application running on the published URL.
- Include your GitHub repo; if not submitted to GitHub, it will result in a 5% off your assignment.
- Provide a detailed YouTube video going over the complete assignment, remember your lecturer might not be able to connect to the same database as yours, so please make sure it is very detailed and shows what is required in the rubric and brief in your video.
- Showing you proof you dropped all your resources

[TOTAL MARKS: 300]

© The Independent Institute of Education (Pty) Ltd 2026

Assessment Sheet (Marking Rubric)

Please note: Tear off this section and **attach** it to your work when you submit it/ If this is an online submission, then this information needs to be included in the online submission.

MODULE NAME:	**MODULE CODE:**
CLOUD DEVELOPMENT A	**CLDV6211**

STUDENT NAME:
STUDENT NUMBER:

Marking Criteria	**Does not meet the required standard**	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
PART 1					
A. Database Design (ERD & Script) **[25 Marks]**	• ERD or Script is missing or completely incorrect.	• ERD is present but incomplete or incorrect. • Script has errors or does not run.	• ERD includes main entities but lacks minor details (e.g., missing keys or cardinality). • Script creates tables but may miss some constraints (PK/FK).	• ERD is professional and complete with all entities (Venue, Event, Booking), relationships, keys, and multiplicities correctly labelled. • Database script (or Migration snapshot) creates tables successfully with correct data types and constraints.	
	0 – 10 Marks	**11-15 Marks**	**16-20 Marks**	**21-25 Marks**	

© The Independent Institute of Education (Pty) Ltd 2026

Marking Criteria	Does not meet the required standard	Meets the required standard	Partially exceeds the required standard	Greatly exceeds the required standard	Feedback
PART 1					
B. MVC Application Structure **[25 Marks]**	• Project does not build or lacks a basic MVC structure.	• Major components (e.g., a Controller) are missing. • Code structure is messy or hard to follow.	• Models, Views, and Controllers are present but may have minor naming or structural issues.	• Solution is well-structured. All Models, Views, and Controllers are present and correctly named. • Code follows standard naming conventions. The 'Code-First' or 'Db-First' approach is evident and correctly implemented.	
	0 – 10 Marks	**11-15 Marks**	**16-20 Marks**	**21-25 Marks**	
C. Functionality & Local Persistence **[25 Marks]**	• Application crashes on run or does not connect to a database.	• CRUD is incomplete (e.g., can Create but not Delete). • Issues with local data persistence.	• CRUD works but has minor bugs. • Data persists, but the connection string setup is hardcoded or messy.	• Full CRUD functionality works seamlessly for Venues and Events. Data persists in **SQL LocalDB** after app restart. • Connection strings in appsettings.json are correctly configured for local development.	
	0 – 10 Marks	**11-15 Marks**	**16-20 Marks**	**21-25 Marks**	

© The Independent Institute of Education (Pty) Ltd 2026

Marking Criteria	Does not meet the required standard	Meets the required standard	Partially exceeds the required standard	Greatly exceeds the required standard	Feedback
PART 1					
D. Cloud Theory **[25 Marks]**	• Theory answers are missing or irrelevant.	• Basic understanding shown, but explanations are vague or contain inaccuracies.	• Good comparison but lacks depth or specific examples. • Definitions are correct but generic.	• Insightful comparison between On-Premises vs. Cloud (Security, Speed, Resources). • Clear understanding of IaaS vs. PaaS vs. SaaS with relevant examples for EventEase.	
	0 – 10 Marks	**11-15 Marks**	**16-20 Marks**	**21-25 Marks**	
No GitHub used and submitted. **[-5 Marks]**	• No GitHub used				
	-5 Marks				

© The Independent Institute of Education (Pty) Ltd 2026

Marking Criteria	Does not meet the required standard	Meets the required standard	Partially exceeds the required standard	Greatly exceeds the required standard	Feedback
PART 2					
A. Local Blob Storage (Azurite) **[25 Marks]**	• No image handling implemented	• Images are not stored in Blob storage (saved to disk instead), or students used live Azure credits prematurely.	• Images upload but may have issues (e.g. wrong file types). • Connected to Azurite, but evidence/screenshots are unclear.	• Image upload functionality works perfectly. appsettings.json uses UseDevelopmentStorage=true. • Evidence (Screenshots) provided of **Azure Storage Explorer** showing the local container and blobs.	
	0 – 10 Marks	**11-15 Marks**	**16-20 Marks**	**21-25 Marks**	
B. Error Handling & Validation **[25 Marks]**	• No validation implemented. The system allows double bookings or crashes.	• Basic validation exists but is easily bypassed. • App crashes on bad input.	• Double booking logic exists but has edge cases. • Validation prevents deletion, but error messages are unclear.	• Robust logic prevents double bookings. • Deletion of active venues/events is blocked with a clear user alert. • App handles invalid inputs gracefully without crashing.	
	0 – 10 Marks	**11-15 Marks**	**16-20 Marks**	**21-25 Marks**	
C. Enhanced Display & Search **[25 Marks]**	• No enhanced view or search implemented.	• View displays raw IDs instead of names. • Search is missing or non-functional.	• Booking view exists, but the layout is poor • Search works but is case-sensitive or buggy.	• Consolidated Booking View is user-friendly and displays data	

© The Independent Institute of Education (Pty) Ltd 2026

Marking Criteria	Does not meet the required standard	Meets the required standard	Partially exceeds the required standard	Greatly exceeds the required standard	Feedback
PART 2					
				from joined tables (Event/Venue names). • Search works accurately for BookingID and Event Name.	
	0 – 10 Marks	**11-15 Marks**	**16-20 Marks**	**21-25 Marks**	
D. Theory: Search & Normalisation **[25 Marks]**	• Theory answers are missing or irrelevant.	• Vague understanding of Cognitive Search. • Normalisation explanation is incorrect.	• Good explanation, but lacks specific use-case examples. • Normalisation discussion is standard but not cloud-specific.	• Clear explanation of Azure Cognitive Search vs Traditional Search. • In-depth discussion on Normalisation in the Cloud (Read vs Write heavy workloads).	
	0 – 10 Marks	**11-15 Marks**	**16-20 Marks**	**21-25 Marks**	
No GitHub used and submitted. **[-5 Marks]**	• No GitHub used				
	-5 Marks				

© The Independent Institute of Education (Pty) Ltd 2026

Marking Criteria	Does not meet the required standard	Meets the required standard	Partially exceeds the required standard	Greatly exceeds the required standard	Feedback
POE PART 3					
A. Advanced Filtering **[20 Marks]**	• Filtering is missing or hardcoded.	• Only one filter works, or logic is flawed (e.g. date range doesn't work).	• Filters work individually but fail when combined. • UI is functional but clunky.	• Filters for EventType, Date Range, and Availability work perfectly together. • UI is intuitive. 'EventType' is correctly implemented as a lookup table/enum.	
	0 – 5 Marks	**6-10 Marks**	**11-15 Marks**	**16-20 Marks**	
B. The Migration (Local to Cloud) **[20 Marks]**	• No attempt to migrate. Still running on localhost.	• Database is on Azure, but App still tries to look at LocalDB (or vice versa).	• Migration was successful, but data was lost (tables were empty). • Config works, but debug keys are left in code.	• **Critical:** Successful migration of Schema and Data from LocalDB to Azure SQL. • Images migrate (or the container is recreated) on Azure Storage. • App configuration is correctly updated to Production keys.	
	0 – 5 Marks	**6-10 Marks**	**11-15 Marks**	**16-20 Marks**	

© The Independent Institute of Education (Pty) Ltd 2026

C. Live Deployment and dropping of resources **[20 Marks]**	• Link returns 404, or the site does not load.	• Site loads but crashes on database access (firewall/connection string issues).	• Site loads but has minor errors (e.g. images don't load due to permission issues).	• Web App is accessible via the Azure URL. The site loads quickly, with no "Server Errors" on startup. • All features (Search, Booking, Images) work in the live environment. • Resources are dropped, and proof is provided	
	0 – 5 Marks	**6-10 Marks**	**11-15 Marks**	**16-20 Marks**	
D. Reflective Report: Technical **[20 Marks]**	• Report is missing or purely descriptive without reflection.	• Report is generic. Lists features but lacks technical reflection on the Azure components.	• Feature list is complete. Migration discussion is present but superficial ("It was hard").	• Comprehensive list of features. Insightful discussion on the "Migration Experience" and challenges faced moving from Emulator to Live. • clear justification of Azure services used.	
	0 – 5 Marks	**6-10 Marks**	**11-15 Marks**	**16-20 Marks**	

© The Independent Institute of Education (Pty) Ltd 2026

E. Theory: Advanced Cloud **[20 Marks]**	• Theory answers are missing or irrelevant.	Vague answers. Confuses SQL/NoSQL concepts.	• Good discussion, but some concepts (e.g. Event Grid) are misunderstood.	• Detailed discussion on CosmosDB vs SQL, Logic Apps security, and Event Grid workflows. • Demonstrates deep understanding of serverless concepts.	
	0 – 5 Marks	**6-10 Marks**	**11-15 Marks**	**16-20 Marks**	
No GitHub used and submitted. **[-5 Marks]**	• No GitHub used				
	-5 Marks				

[TOTAL MARKS:300]

© The Independent Institute of Education (Pty) Ltd 2026